

GUIMigrator: Semantics-Preserving Transpilation from Android XML to Compose and SwiftUI

YI GAO, Zhejiang University, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

XING HU, Zhejiang University, China

XIAOHU YANG, Zhejiang University, China

XIN XIA*, Zhejiang University, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

Constructing user interfaces (UIs) is one of the most resource-intensive tasks in mobile development, often consuming more than half of overall effort. Although declarative frameworks such as Jetpack Compose (Android) and SwiftUI (iOS) have become mainstream, the majority of existing Android apps still rely on legacy XML-based layouts. Migrating these UIs to declarative paradigms is essential for maintainability and cross-platform reuse, but manual migration is costly, error-prone, and difficult to scale. We present GUIMIGRATOR, a semantics-preserving framework that automates the migration of Android XML-based UIs to Jetpack Compose and SwiftUI. We design the *Semantic UI Transpiler (SUT)*, which abstracts layout structures and resource semantics from legacy XML and systematically re-expresses them using the component abstractions and idioms of modern declarative frameworks. This design ensures that migrated UIs preserve both visual fidelity and functional equivalence, while generating idiomatic, compilable code that maintains cross-platform consistency with minimal manual intervention. By separating semantic interpretation from platform-specific realization, GUIMIGRATOR provides a deterministic yet extensible basis for cross-platform modernization, avoiding the unpredictability of purely generative approaches. We evaluate GUIMIGRATOR on 31 open-source applications across ten domains. Results show that GUIMIGRATOR achieves high migration completeness and strong visual similarity (81.9% SSIM on Jetpack Compose and 78.2% on SwiftUI on average), while maintaining substantially higher project-wide semantic coherence (PSC) than modern LLM baselines. In addition, GUIMIGRATOR reduces manual development effort by over 90%. These findings demonstrate that GUIMIGRATOR provides an effective and practical solution for reusing Android UIs across modern declarative frameworks, advancing automated cross-platform UI development.

CCS Concepts: • **Software and its engineering** → **Software evolution; Reusability.**

Additional Key Words and Phrases: UI Transpiler, UI Migration, SwiftUI, Jetpack Compose, Android, iOS

ACM Reference Format:

Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. GUIMigrator: Semantics-Preserving Transpilation from Android XML to Compose and SwiftUI. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE208 (July 2026), 23 pages. <https://doi.org/10.1145/3808215>

*Corresponding Author

Authors' Contact Information: Yi Gao, Zhejiang University, Hangzhou, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, gaoyi01@zju.edu.cn; Xing Hu, Zhejiang University, Hangzhou, China, xinghu@zju.edu.cn; Xiaohu Yang, Zhejiang University, Hangzhou, China, yangxh@zju.edu.cn; Xin Xia, Zhejiang University, Hangzhou, China and the Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hangzhou, China, xin.xia@acm.org.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE208

<https://doi.org/10.1145/3808215>

1 Introduction

Mobile applications (*apps*) have become an indispensable part of modern life, supporting communication, commerce, entertainment, and productivity [18, 23]. To improve user experiences, companies usually provide *apps* with attractive and consistent UIs on different platforms [13, 21, 27, 28, 40, 45, 51], such as Android [1] and iOS [8]. However, UI development is among the most resource-intensive tasks, often consuming over half of the total engineering effort [30]. Recently, declarative frameworks such as Jetpack Compose (Android)[4] and SwiftUI (iOS)[12] have been promoted as the standard for maintainable and expressive UIs. These frameworks represent a paradigm shift from legacy XML layouts, which still dominate many Android *apps*. As mobile ecosystems evolve, migrating legacy XML UIs to modern declarative paradigms is essential for maintainability and reuse. However, manual migration is costly, error-prone, and impractical at scale—especially for organizations maintaining large, long-lived Android codebases [26].

Several research and industrial efforts have attempted to reduce the burden of UI development [14, 18, 33, 39, 48, 50, 53]. Computer-vision-based approaches translate UI screenshots into code, but they are fragile to minor variations: even small changes in resolution, style, or occlusion often lead to misrecognized elements and unusable layouts. Cross-platform frameworks such as Flutter [5] and React Native [11] mitigate duplication by enabling code reuse, but they require developers to redesign UIs in an entirely different paradigm, sacrificing native fidelity and breaking compatibility with existing codebases. More recently, large language models (LLMs) have been explored for code translation, showing promise in generating declarative UI code. Nevertheless, LLMs frequently hallucinate non-existent components, misorder attributes, or generate unstable code that fails to compile—yielding results that are neither predictable nor reproducible.

Migrating legacy Android XML layouts into modern declarative frameworks is a necessary but non-trivial task. The task is complicated by several semantic gaps between paradigms: First, *Language heterogeneity*: Android XML encodes UIs in an imperative, tree-structured format, while declarative frameworks (Compose, SwiftUI) rely on composable functions and nested containers. Direct syntactic translation fails to preserve layout intent. Second, *Resource incompatibility*: Android resources such as colors, fonts, and vector graphics do not directly map to iOS or Compose counterparts, requiring systematic adaptation rather than simple substitution. Finally, *Complex layouts*: Advanced constructs like `ConstraintLayout` encode relational constraints among views that lack direct equivalents in SwiftUI, demanding careful semantic approximation to preserve visual fidelity. These challenges reveal that UI migration cannot be solved by syntactic rewriting alone. Instead, it requires a semantics-preserving transformation that bridges structural, visual, and resource-level differences while producing idiomatic code for the target platform.

Our Insight. Despite the syntactic and semantic differences, declarative frameworks such as Jetpack Compose and SwiftUI provide comparable abstractions for layout, views, and resources. This observation suggests that existing Android XML-based UIs can be systematically reused by migrating them into equivalent declarative constructs, provided that the migration process captures layout semantics and adapts resources appropriately.

In this paper, we present GUIMIGRATOR, a semantics-preserving approach for migrating legacy Android XML-based UIs to modern declarative frameworks. GUIMIGRATOR formulates UI migration as a semantic transpilation problem across heterogeneous UI paradigms. GUIMIGRATOR represents a semantic-to-declarative migration methodology that bridges imperative XML-based layout semantics and declarative UI frameworks through a reusable semantic layer, enabling cross-paradigm migration without manual one-to-one widget mappings. At its core is the *Semantic UI Transpiler (SUT)*, which constructs a platform-agnostic intermediate representation that encodes both structural hierarchies and semantic relationships extracted from Android XML. GUIMIGRATOR performs

migration through a structured three-step pipeline: ❶ **Resource Parsing and Adaptation**, which resolves and normalizes Android-specific assets; ❷ **UI Transpilation**, which interprets the semantic roles of layouts and views and maps them into equivalent declarative constructs; and ❸ **Target UI Generation**, which instantiates the transpiled models into platform-specific code templates, producing compilable SwiftUI and Jetpack Compose code.

By following this structured pipeline, GUIMIGRATOR enables efficient and accurate modernization of legacy Android UIs while reducing the manual burden of redeveloping UIs across platforms. This new paradigm separates UI semantic modeling from platform-specific realization, providing a deterministic and extensible way to preserve intent across heterogeneous frameworks while avoiding the unpredictability of LLM-based generation and the redesign cost of cross-platform frameworks.

We evaluate GUIMIGRATOR on 31 open-source Android applications spanning ten domains. Our study measures migration quality in terms of correctly transpiled layouts and views, visual fidelity between pre- and post-migration screenshots, and code correctness. Across 1,027 XML layouts, GUIMIGRATOR achieves substantially higher similarity than GPT-5.2 and DeepSeek-3.2 baselines, while maintaining stable and compilable code. A controlled user study further confirms that GUIMIGRATOR reduces developer effort by over an order of magnitude in both code changes and development time compared with manual implementation. Finally, performance measurements show that the entire dataset can be migrated within a few seconds, demonstrating the efficiency and practicality of our approach. This work makes the following contributions:

- We propose the first semantics-preserving UI migration method that formulates legacy Android XML layouts as a semantic transpilation problem, enabling reusable migration across modern declarative frameworks (Jetpack Compose and SwiftUI).
- We introduce a semantic transpilation methodology based on the Semantic UI Transpiler (SUT), which captures cross-paradigm UI semantic invariants and models platform-agnostic layout semantics, enabling systematic translation into platform-specific declarative UI code.
- We evaluate GUIMIGRATOR on 31 open-source applications from ten domains. Results show that GUIMIGRATOR achieves high migration completeness and visual fidelity, outperforms GPT-5.2 and DeepSeek-V3.2 baselines, and reduces manual development effort by over an order of magnitude.

2 Motivation Example

Use Case. Consider a mobile development company that wishes to modernize its legacy Android applications. Most of its UIs are defined in XML, but the company now wants to (i) adopt Jetpack Compose for new Android versions and (ii) support iOS with SwiftUI to reach more users. The straightforward option is to rebuild UIs manually, requiring developers proficient in both platforms. Alternatively, cross-platform frameworks (e.g., Flutter, React Native) provide portability but sacrifice native fidelity and introduce performance overhead. Recent approaches leveraging screenshots or LLMs can partially reuse existing designs, but they often fail to respect semantic UI constraints, resulting in inaccurate layouts or hallucinated code. These limitations make the direct reuse of legacy Android UIs across platforms both costly and unreliable.

Example. We illustrate this challenge with the login page of *OpenShop* [10], a representative open-source Android app. As shown in Figure 1, the interface combines multiple nested layouts (RelativeLayout and LinearLayout), text prompts, and styled buttons. To migrate such a design, developers must resolve three non-trivial issues: (1) *structural heterogeneity*, since Android XML expresses layouts imperatively while SwiftUI/Compose adopt declarative paradigms; (2) *resource incompatibility*, as color, dimension, and vector formats differ across platforms; and (3) *constraint simulation*, because constructs like ConstraintLayout have no direct counterpart in SwiftUI.

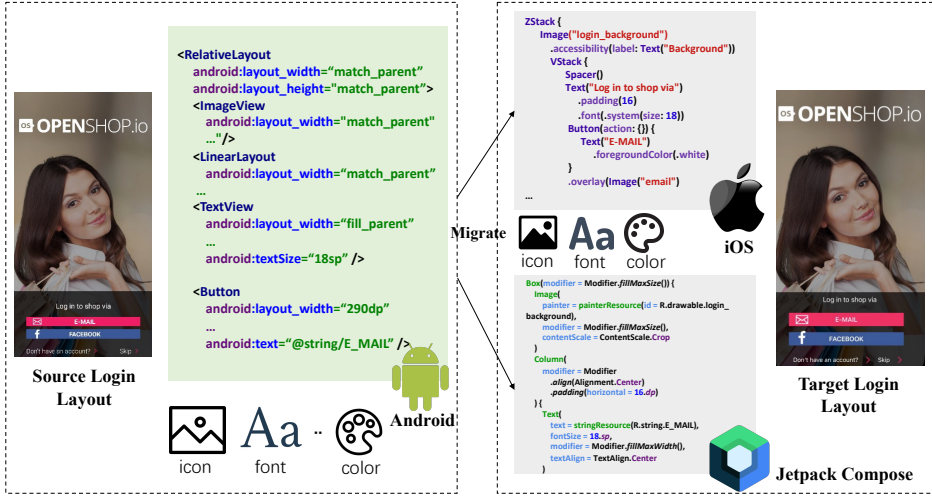


Fig. 1. Example of cross-platform UI migration. A legacy Android XML layout is converted into SwiftUI (iOS) and Jetpack Compose (Android), illustrating the structural gap between imperative XML and declarative frameworks and motivating a semantics-preserving transpilation approach.

Motivation. This example highlights the semantic gap between legacy Android XML and modern declarative frameworks. Naïve syntactic translation cannot guarantee visual or behavioral fidelity, and existing alternatives either require significant manual effort or fail to preserve UI semantics. Therefore, an automated approach that operates on a *semantic representation of UI structures and resources*, and systematically bridges the gap between XML and declarative paradigms, is needed. This motivates the design of GUIMIGRATOR, which transpiles Android UIs into SwiftUI and Jetpack Compose while preserving both structural and visual semantics.

3 Problem Definition

We model legacy Android UIs as structured artifacts and define the migration task as a *semantics-preserving transformation* that bridges heterogeneous UI paradigms.

Input. An Android UI is denoted as $U_A = (L, V, R)$, where L is the set of layout containers, V the set of views, and R the set of resources (e.g., colors, strings, images). Together, these elements encode both the structural hierarchy of the interface and the assets that ensure visual consistency.

Output. The goal is to produce a target declarative UI $U_T = (L', V', R')$ in Jetpack Compose or SwiftUI, which realizes the same layout intent and visual semantics while conforming to the idioms of the target framework.

UI Semantic Graph. We abstract U_A into a graph $\mathcal{G}_A = (N_A, E_A)$, where nodes $N_A = L \cup V \cup R$ and edges E_A capture three core relations: (i) containment $\text{contains}(x, y)$, representing structural nesting; (ii) alignment and constraints $\text{align}(x, y, \tau)$, representing spatial relations among elements; and (iii) resource dependencies $\text{uses}(x, r)$, linking UI elements to external assets. The target UI U_T is expressed as an analogous graph $\mathcal{G}_T$, enabling direct reasoning about structural and semantic correspondence.

Problem Statement. The migration task is to compute a mapping $h : \mathcal{G}_A \rightarrow \mathcal{G}_T$ and a resource adaptation $\rho : R \rightarrow R'$ that jointly satisfy:

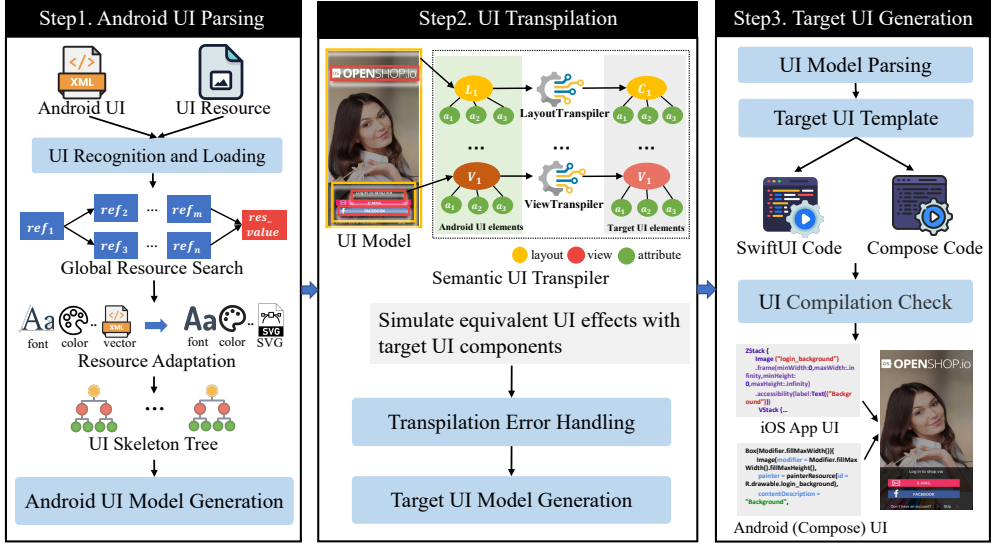


Fig. 2. Overview of the GUIMIGRATOR pipeline.

- C1 Structural preservation:** the hierarchical relations among layouts and views should be maintained or consistently approximated;
- C2 Visual fidelity:** the rendered appearance of the target UI should closely resemble the source, as measured by perceptual similarity metrics;
- C3 Semantic consistency:** essential attributes (e.g., size, color, text) should be adapted in a way that preserves their intended meaning.

Due to heterogeneous syntaxes, non-isomorphic layouts (e.g., `ConstraintLayout`), and divergent resource formats, exact isomorphism is infeasible. Hence, the migration requires a semantic mapping strategy that interprets structural and visual intent at an intermediate level and reconstructs equivalent declarative constructs in the target framework.

4 Proposed Approach

Figure 2 presents the overall framework of GUIMIGRATOR. Starting from legacy Android XML UIs, our approach (i) parses layouts, views, and resources into a unified *UI Semantic Graph*, (ii) applies semantic transpilation to map Android constructs into declarative counterparts, and (iii) generates compilable SwiftUI and Jetpack Compose code through platform-specific templates. This pipeline ensures that migration is both semantically faithful and practically executable.

4.1 Android UI Parsing

The first step is to extract Android UI elements and represent them in a form suitable for semantic transformation. Rather than operating directly on XML syntax, we construct an intermediate abstraction that captures the structure and attributes of legacy UIs in a platform-independent manner, forming the basis of the *UI Semantic Graph* (Section 3). Formally, we model an Android project as $U_A = (L, V, R)$, where L , V , and R denote layouts, views, and resources defined under the `res` directory. Dynamic components (e.g., Activity classes) are excluded, since they are written in Java/Kotlin and primarily encode business logic rather than declarative UI specifications. Building on this abstraction, we define the *UI Semantic Graph* as $\mathcal{G}_A = (N_A, E_A)$, where nodes N_A

Table 1. Structure of the Android *res* directory and representative UI resource types. These resources are parsed and normalized as the first step of migration.

Type	Example Files	Description
Layout	res/layout/main.xml, res/layout/dashboard.xml	XML files that define screen layouts and hierarchical UI structures.
Drawable	res/drawable/icon.png, res/drawable/background.xml	Raster images and XML graphics used for icons and backgrounds.
String	res/values/strings.xml	Localized text resources displayed in the interface.
Color	res/values/colors.xml	Color definitions referenced by layouts and styles.
Dimension	res/values/dimens.xml	Values (e.g., margins, font sizes) ensuring consistent sizing across screens.
Style/Theme	res/values/styles.xml	Themes and style definitions that standardize visual appearance.
Vector	res/drawable/ic_launcher.xml	Scalable vector graphics for resolution-independent icons.

correspond to elements in $L \cup V \cup R$ and edges E_A encode containment and reference relations. This formalism provides a uniform representation that we use consistently in Section 4.2 to describe semantics-preserving transformations.

Parsing is challenging for three reasons. First, resources often involve multiple layers of indirection (e.g., `@color/primary` $\rightarrow$ `@android:color/holo_blue`). Second, layouts rely on heterogeneous managers such as `LinearLayout`, `ConstraintLayout`, and `RelativeLayout`, which encode structure differently. Third, attribute naming and value domains are inconsistent (e.g., dimensions may be expressed in `dp`, `px`, or `sp`). Our parser therefore (a) extracts layouts and views into a hierarchical tree reflecting containment, (b) resolves resource references recursively through global lookup until concrete values are obtained, and (c) normalizes attributes (e.g., converting `16dp`, `16px`, and `16sp` into a uniform numeric representation). For example, given a snippet `<TextView android:textColor="@color/primary"/>`, if `primary` is defined as `@android:color/holo_blue`, our parser dereferences both layers and normalizes the final value into the canonical RGB representation.

The output is a rooted forest where each node corresponds to a UI element x with attributes $\alpha(x)$ and edges encode containment or resource usage. This forest is then embedded into the UI Semantic Graph $\mathcal{G}_A = (N_A, E_A)$, which provides a uniform semantic representation for downstream transpilation. Unlike traditional XML parsers, this semantic graph abstraction makes structural, visual, and resource information explicit, enabling systematic reasoning and facilitating generalization beyond Android-specific syntax.

4.1.1 UI Recognition and Loading. Android projects follow a standardized structure in which user interface (UI) resources are organized under the `res` directory. As shown in Table 1, this directory contains layouts, drawables, strings, colors, dimensions, and styles that collectively define the static aspects of an app's UI. Among them, layout XML files specify the hierarchical arrangement of views, while auxiliary resources (e.g., colors, dimensions, styles) provide reusable definitions that ensure visual consistency.

To prepare for migration, we parse all XML layouts together with their referenced resources and represent them in a platform-independent form. Each layout is transformed into a hierarchical

tree that captures view containment and attributes, while resource references are linked to their definitions rather than treated as raw symbols. This representation yields a complete inventory of the app's static UI elements, abstracted away from Android-specific syntax, and serves as the entry point for constructing the UI Semantic Graph used in subsequent transpilation.

4.1.2 UI Resource Normalization. Android XML layouts frequently reference external resources such as colors, dimensions, or styles, which may themselves be defined through multiple layers of indirection. While this indirection improves modularity in Android, it complicates migration because declarative frameworks (e.g., Jetpack Compose, SwiftUI) expect explicit values instead of symbolic references. For example, `@color/textColor` may first resolve to `@color/account` before eventually mapping to a concrete hex value.

To address this, we design a *Global Resource Search (GRS)* mechanism that recursively resolves resource references until concrete values are obtained and substitutes them directly into the parsed UI tree. Starting from a resource reference (e.g., `@color/textColor`), GRS recursively follows indirections in Android resource tables (including `colors.xml`, `dimens.xml`, `strings.xml`, and `drawable selectors`) until a terminal value is reached. To ensure robustness, GRS maintains a visited set and enforces a maximum resolution depth to prevent cyclic dependencies and infinite expansion. If no terminal value can be resolved, the original reference is preserved and marked as unresolved for backend fallback handling. Through GRS, each UI element in the semantic UI graph is associated with an explicit, platform-agnostic representation of its attributes (e.g., normalized RGB color values or pixel-level dimensions).

Beyond resolution, Android and iOS also differ in resource formats and data models, making direct reuse infeasible. We therefore introduce a *UI Resource Adapter* that maps normalized values into target-specific constructs. Instead of directly mapping Android resources to platform APIs, the adapter applies *typed adaptation rules* that interpret each resource according to its semantic role (e.g., color, dimension, text, or image) and translates it into a platform-agnostic representation. These canonical forms (e.g., *fill*, *wrap*, or *fixed* sizing semantics, resolved color values, or abstract image identifiers) are then instantiated by SwiftUI or Jetpack Compose. In our implementation, the adapter covers four categories of adaptations (color, dimension, textual, and drawable resources), which account for the dominant resource usages observed in modern Android apps. For example, Android hexadecimal colors are converted into SwiftUI's Color initializers, while vector drawables are transformed into portable formats such as SVG or PDF. In Jetpack Compose migration, the same normalized values are injected into APIs like `colorResource` or `painterResource`. Together, GRS and the adapter implement a semantic normalization layer that consolidates heterogeneous Android resources into a uniform, cross-platform representation, enabling systematic and faithful transpilation to modern declarative frameworks.

In our implementation, the adapter includes four categories of adaptations—color, dimension, textual, and drawable resources—covering the dominant resource usages observed in modern Android apps. Together, GRS and the adapter form a semantic normalization layer that consolidates heterogeneous Android resource definitions into a uniform cross-platform representation, enabling deterministic and consistent transpilation to modern declarative UI frameworks.

4.1.3 Android UI Model Generation. Android UIs consist of heterogeneous elements—layouts, views, and resources—connected by containment and reference relations. Accurately capturing these relationships is essential for migration but challenging due to nested hierarchies and indirect bindings. To this end, we first construct an Android UI Model—a structured tree rooted at the top-level layout, where internal nodes represent nested containers and leaves correspond to views (e.g., `Button`, `TextView`). Each node is annotated with resolved attributes and normalized resources, yielding a context-rich abstraction that captures both the structure and semantics of the

original XML. This model is then lifted into the UI Semantic Graph, which generalizes Android-specific constructs into a platform-independent representation suitable for systematic reasoning and semantics-preserving transformation.

This model serves as the *input* to the UI Semantic Graph. Concretely, the model captures the Android-specific hierarchy in a structured yet platform-dependent form, whereas the UI Semantic Graph generalizes it into a platform-independent graph representation that explicitly encodes structural, visual, and resource semantics. In other words, the model is a faithful semantic snapshot of an Android layout, and the graph lifts this snapshot into a uniform abstraction suitable for systematic reasoning and cross-platform transpilation.

4.2 UI Transpilation

The goal of this step is to transform the Android UI model into target declarative representations while preserving semantic fidelity. Direct syntactic translation from imperative XML to declarative frameworks such as SwiftUI or Jetpack Compose is infeasible: Android relies on nested imperative layouts (e.g., `RelativeLayout`, `ConstraintLayout`), whereas declarative frameworks describe composition through functional containers (e.g., `VStack`, `ZStack`, `Row`, `Box`). This structural mismatch, together with divergent resource systems, renders one-to-one mappings insufficient.

To address this, we design a *Semantic UI Transpiler (SUT)* that operates at the semantic-level transformation. SUT takes the Android UI model (Section 4.1) as input and produces a target representation through three steps: **❶ Semantic interpretation.** Each layout or view is analyzed in terms of its functional role (e.g., sequential arrangement, overlapping composition, text rendering) rather than its literal XML tag. **❷ Semantic-to-declarative mapping.** A set of parametric transformation schemas maps these semantic roles into equivalent declarative constructs. For example, a horizontal `LinearLayout` corresponds to a `Row` in Compose or an `HStack` in SwiftUI, while a `ConstraintLayout` is decomposed into nested stacks augmented with alignment modifiers. **❸ Layout reconstruction.** The target layout is synthesized by composing transformed nodes while preserving containment, alignment, and resource bindings. This reconstruction step enforces both structural coherence and idiomatic usage of the target framework.

By reconstructing UIs from semantic roles instead of performing literal XML translations, SUT bridges heterogeneous UI paradigms and supports both cross-platform migration (Android $\rightarrow$ iOS) and intra-platform modernization (XML $\rightarrow$ Compose). Rather than replicating Android-specific layouts, GUIMIGRATOR preserves UI semantics and realizes them using the native declarative layout systems of the target platform. Layout attributes such as padding and spacing are interpreted as relative layout semantics instead of fixed coordinates, and are therefore resolved by SwiftUI or Jetpack Compose through their device adaptive layout mechanisms. This allows structural intent (e.g., containment, alignment, and spacing) to be preserved while naturally conforming to platform conventions such as screen-size adaptation, without enforcing platform-specific aesthetic redesign. More broadly, SUT exemplifies a paradigm of *semantics-guided graph transformation*, which can generalize to other software migration tasks where heterogeneous syntaxes must be reconciled through an intermediate semantic abstraction.

4.2.1 UI Transpilation Process. As illustrated in Figure 2, the *Semantic UI Transpiler (SUT)* converts the Android UI model into target declarative structures through a semantics-guided graph transformation. Each XML layout is first represented as a rooted tree, with layout containers as internal nodes and views as leaves. SUT traverses this tree and interprets each node in terms of its semantic role (e.g., sequential arrangement, overlay, relative positioning), before reconstructing an equivalent declarative tree in SwiftUI or Jetpack Compose.

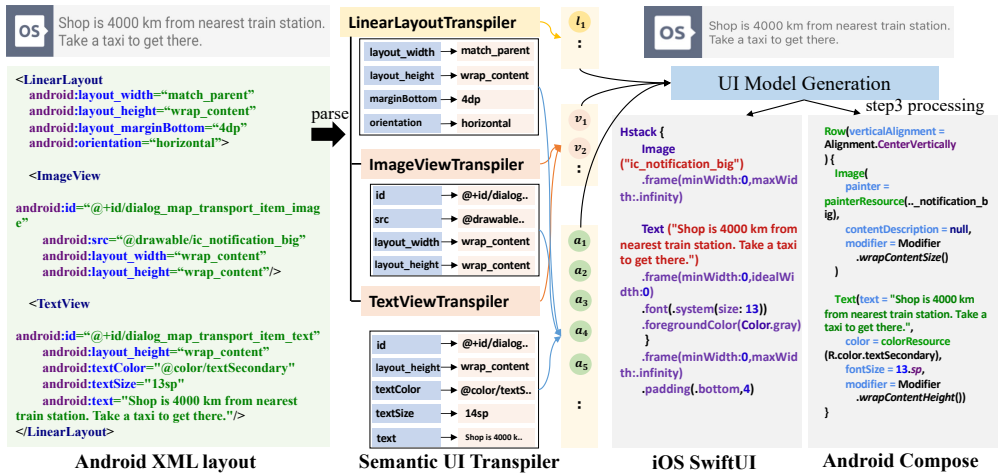


Fig. 3. An example of transpiling an Android UI layout to an equivalent visual interface using SUT. The figure shows how structural hierarchies and semantic roles in the source XML are interpreted and reconstructed into declarative constructs, preserving layout intent while adapting to the target platform.

For simple layouts, the mapping is direct: a horizontal `LinearLayout` corresponds to `HStack` (SwiftUI) or `Row` (Compose), while a vertical one maps to `VStack` or `Column`. View components are similarly mapped, such as `ImageView`→`Image` and `TextView`→`Text`. Because both SwiftUI and Compose share declarative paradigms, SUT applies the same semantic mappings across platforms, with only minor syntactic adaptations. The main challenge lies in constructs like `ConstraintLayout`, which encode relational constraints that are neither directly supported in SwiftUI nor idiomatic in modern Compose. SUT handles complex layouts such as `ConstraintLayout` by *semantic decomposition*. Instead of enumerating low-level constraints, SUT abstracts them into relative positioning semantics (e.g., align-to-parent, align-to-sibling). These semantics are then systematically realized via nested declarative containers (`ZStack/Box`, `VStack/Column`, `HStack/Row`) with alignment modifiers, padding, and spacers. This abstraction layer ensures that even intricate constraints are faithfully reconstructed in an idiomatic style.

As illustrated in Figure 4, an Android `ConstraintLayout` may specify relative positioning rules such as `layout_constraintBottom_toBottomOf` or `layout_constraintStart_toStartOf`. Rather than directly encoding these constraints, SUT decomposes them into equivalent declarative structures: for SwiftUI, constraints are mapped to compositions of `ZStack`, `VStack`, and `HStack` with explicit alignment and spacing; for Jetpack Compose, analogous effects are achieved with `Row`, `Column`, and `Spacer`. This decomposition preserves the original relative positioning semantics (e.g., *align to parent bottom*, *center horizontally within parent*) while producing idiomatic target code that aligns with the design philosophy of declarative frameworks. By eliminating low-level constraint specifications and replacing them with higher-level semantic containers, the generated code is not only visually faithful but also more maintainable and extensible in the target platform.

Extensibility. Real-world Android *apps* typically rely on a recurring set of standard layouts and views. SUT currently supports 30 component types and 75 attributes, which already cover the majority of practical cases. To remain adaptable to evolving UI frameworks, SUT adopts a *semantic plugin architecture*: new components are added by declaring their semantic roles and associating them with target constructs. This modular design ensures that extending support to additional layouts or attributes requires minimal effort and preserves maintainability over time.

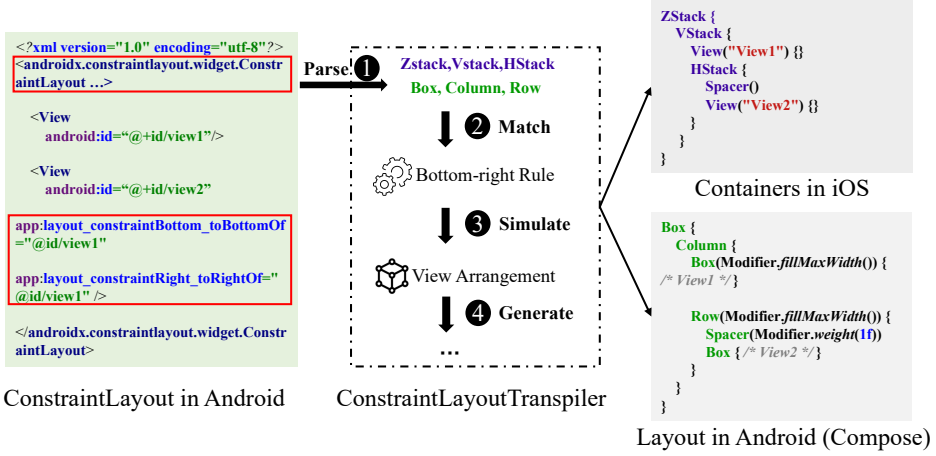


Fig. 4. Transpilation process of converting Android's `ConstraintLayout` to visually equivalent container combinations using SUT.

Error Handling. In practice, migration often encounters components or attributes without direct counterparts on the target platform. SUT ensures robustness through a two-level fallback strategy: (i) unrecognized layouts are replaced with generic containers that preserve structural relations, and (ii) unknown views are substituted by placeholders annotated with their key attributes (e.g., size, position, color). This guarantees syntactic validity and preserves high-level visual intent, even in the absence of exact mappings. In addition, SUT generates a detailed migration report that records unresolved elements and the reasons for failure, enabling developers to refine transpilation or integrate custom plugins. This mechanism not only accommodates third-party components but also illustrates a broader methodological principle: *semantic degradation with traceability*, where missing constructs are gracefully approximated while maintaining developer visibility and control.

4.2.2 Target UI Model Generation. The output of SUT is a structured representation of the target UI, referred to as the *Target UI Model*. Each model encodes container types (e.g., `ZStack`, `VStack`), associated attributes, and nested views, forming a hierarchical abstraction of the interface. This intermediate representation is platform-neutral: it suppresses syntactic details while retaining all semantic information needed for code realization. By aggregating such models, we obtain a complete, declarative description of the target application's UI.

4.3 Target UI Generation

The final step translates Target UI Models into executable SwiftUI or Jetpack Compose code. The objective is not only to produce compilable source files but also to ensure that the code is *idiomatic*, preserving both semantic fidelity and target-specific conventions.

4.3.1 UI Model Parsing. Before code emission, we perform platform-specific validation on the Target UI Models. Declarative frameworks impose subtle ordering and scoping rules that directly impact rendering and compilation. For SwiftUI, for example, modifier ordering is crucial: frame must precede background to ensure correct sizing, while padding must be placed before alignment modifiers to avoid layout inconsistencies. Jetpack Compose enforces similar rules through chained modifier functions, where improper ordering can lead to visually incorrect layouts.

To address this, each Target UI Model is augmented with an ordered modifier list derived from dependency constraints between attributes. For instance, dimension-related attributes are resolved

before styling attributes, and spacing must be placed before alignment. This ordering is enforced during code generation, guaranteeing that the emitted code is both syntactically valid and visually consistent. The validated models thus form a reliable basis for synthesis into executable UI files.

4.3.2 UI Code Generation. Code generation is template-driven but semantically informed. Each Target UI Model node is mapped to a code template parameterized by its attributes and child elements. As shown in Figure 5, for SwiftUI, templates generate struct definitions with a body function and optional PreviewProvider constructs to enable live rendering in Xcode. For Jetpack Compose, the system emits Kotlin @Composable functions annotated with @Preview, ensuring direct integration with Android Studio’s preview tooling.

Beyond direct translation, SUT also injects target-specific idioms to improve readability and maintainability. For example, groups of aligned child views are wrapped in higher-order containers (Group in SwiftUI or Column/Row in Compose), and reusable resources are factored into constants to avoid duplication. We also generate scaffolding code (e.g., import SwiftUI, setContent in Compose) so that the synthesized output is self-contained and immediately compilable.

This design balances automation with human-centric readability: the generated code is not only functionally equivalent but also stylistically aligned with the conventions of SwiftUI and Compose developers, facilitating downstream modification and reuse.

4.3.3 UI Compilation Check. Finally, we validate the generated code through native toolchains to ensure practical usability. SwiftUI code is compiled in Xcode and inspected via its live preview engine, while Compose code is compiled in Android Studio and rendered with Compose Preview. When compilation or rendering fails, GUIMIGRATOR produces an automated migration report that pinpoints the responsible UI elements and attributes, and provides both the failing code and suggested fallback constructs. This mechanism prevents silent failures and shortens the refinement loop for developers, turning migration results into actionable outputs. In this way, GUIMIGRATOR guarantees that the final code is not only semantically faithful but also executable within production environments, completing the end-to-end migration pipeline from Android XML to modern declarative frameworks.

```

Let  $S = \{\text{Struct Definition}\}$ 
  where  $S \ni s = \text{struct fragment\_name: View } \{ \dots \}$ 
Let  $B = \{\text{View Body}\}$ 
  where  $B \ni b = \text{var body: some View } \{ \dots \}$ 
Let  $P = \{\text{Preview Section}\}$ 
  where  $P \ni p = \text{struct fragment\_name\_Previews}$ 
  : PreviewProvider  $\{ \dots \}$ 

Let  $F = \{\text{Composable Function Definition}\}$ 
  where  $F \ni f = \text{@Composable fun FragmentName() } \{ \dots \}$ 
Let  $V = \{\text{View Composition Block}\}$ 
  where  $V \ni v = \{ \dots \}$ , Compose UI elements
Let  $P = \{\text{Preview Annotation Block}\}$ 
  where  $P \ni p = \text{fun FragmentNamePreview() } \{$ 
  FragmentTheme() $\}$ 

```

Fig. 5. SwiftUI and Compose code generation templates. They define how Target UI Models are instantiated into platform-specific constructs (e.g., struct/@Composable), ensuring the generated code is both compilable and idiomatic.

5 Evaluation

We evaluate GUIMIGRATOR through a series of experiments designed to answer the following research questions:

- **RQ1: Migration Quality.** How is the quality of the UI migrated by GUIMIGRATOR?
- **RQ2: Migration Effectiveness.** How effective is GUIMIGRATOR in preserving UI fidelity and reducing developer effort?
- **RQ3: Development Effort Reduction.** How does GUIMIGRATOR reduce manual development effort in UI transpilation?

Table 2. Overview of the Android UI migration dataset across ten domains. We report the number of XML files, layouts, views, and lines of code (LOC), as well as average layouts per XML and views per layout to indicate structural complexity.

Domain	XMLs	Layouts	Views	LOC	Layouts/XML	Views/Layout
Business	57	188	302	4,740	3.30	1.61
Communication	41	84	214	4,029	2.05	2.55
Education	27	36	132	1,893	1.33	3.67
Finance	68	128	376	5,281	1.88	2.94
Health	13	28	143	1,825	2.15	5.11
Media	35	57	121	2,505	1.63	2.12
News	69	128	193	2,419	1.86	1.51
Productivity	29	51	97	1,809	1.76	1.90
Network	44	96	151	2,304	2.18	1.57
Tools	117	231	854	10,612	1.97	3.70
Total	500	1,027	2,583	37,417	2.05	2.52

Role: You are an expert in the field of Android and iOS, specializing in equivalent UI transpilation.
User: Given the following *Android XML code* for a GUI layout, transpile it into an equivalent *SwiftUI or Compose code* that achieves the *same UI effect*.
Original UI: {Android app UI}

Fig. 6. Prompt Template for transpiling Android UI to declarative frameworks (SwiftUI and Jetpack Compose).

• **RQ4: Migration Performance.** What is the runtime efficiency of GUIMIGRATOR in UI migration?

5.1 Experimental Setup

Dataset. Since UI migration requires full access to application resources, we draw candidate projects from the open-source-android-apps collection on GitHub [9], which aggregates widely used open-source Android applications (9.8k stars). To ensure both reliability and diversity, we apply the following criteria: (i) the project must provide complete source code and use Gradle as its build system, ensuring a consistent project structure for resource parsing; and (ii) the project should represent a broad range of domains, allowing us to evaluate the generality of migration across heterogeneous UI designs.

From this pool, we randomly sample 40 projects and verify that they can be successfully built, compiled, and previewed. After excluding those that fail to meet these requirements, we retain 31 applications spanning ten domains (e.g., business, communication, media, social networking). As summarized in Table 2, the dataset covers 500 XML files comprising 1,027 layouts and 2,583 views, totaling 37,417 lines of UI code. This collection captures both simple and complex UI patterns, including nested layouts and resource-heavy designs, providing a realistic benchmark for assessing semantic migration. To promote reproducibility, the complete dataset list is publicly available at [7]. This dataset serves as the foundation for our evaluation of migration completeness, semantic fidelity, and execution robustness in subsequent experiments.

Baseline. To the best of our knowledge, GUIMIGRATOR is the first approach that automates end-to-end migration from Android XML layouts to declarative UI frameworks (SwiftUI and Jetpack Compose). We are not aware of any prior tool that directly supports this task. Therefore, we use the latest proprietary ChatGPT model (GPT-5.2) and the open-source DeepSeek-V3.2 as baselines,

Table 3. Results of UI migration across ten domains. For each category, we report the number of successfully migrated elements (M), the number of omitted elements (O), and the migration rate (%). **Err.** counts syntax errors in generated code.

Domain	Migration to Jetpack Compose										Migration to SwiftUI									
	XML			Layouts			Views			Err.	XML			Layouts			Views			Err.
	M	O	(%)	M	O	(%)	M	O	(%)		M	O	(%)	M	O	(%)	M	O	(%)	
Business	53	4	93	185	3	98	295	7	98	3	54	3	94	186	2	98	297	5	98	5
Comm.	30	11	73	68	16	81	153	61	71	6	30	11	73	68	16	81	153	61	71	8
Education	25	2	92	31	5	86	121	11	92	1	25	2	92	32	4	88	125	7	94	2
Finance	60	8	88	119	9	93	347	29	92	8	56	12	82	113	15	88	310	66	84	11
Health	13	0	100	28	0	100	143	0	100	0	13	0	100	28	0	100	143	0	100	0
Media	32	3	91	55	2	96	115	6	95	1	31	4	88	54	3	94	111	10	91	2
News	67	2	97	126	2	98	188	5	97	4	67	2	97	125	3	97	186	7	96	3
Productivity	27	2	93	49	2	96	92	5	94	7	26	3	89	48	3	94	87	10	89	9
Network	43	1	97	92	4	96	150	1	99	5	42	2	95	90	6	93	149	2	98	2
Tools	110	7	94	228	3	98	810	44	94	8	109	8	93	226	5	97	767	87	89	10
Total	460	40	92.0	981	46	95.5	2414	169	93.5	43	453	47	90.6	970	57	94.5	2328	255	90.1	61

both accessed with default parameters and without fine-tuning. We design a unified prompt that supplies the source Android XML and explicitly requests translation into either SwiftUI or Jetpack Compose, including the target platform and expected output format (Figure 6). To ensure fairness, we avoid task-specific tool augmentation or bespoke post-processing, limiting the interaction to what an end user could reasonably provide in a single prompt.

We do not include screenshot-to-UI generation methods (e.g., [18]) in the main quantitative benchmark, as they address a related but different problem formulation. Such methods take images as input and attempt to reconstruct UI code, focusing primarily on visual similarity. In contrast, our task is *semantics-preserving migration*: the input already contains a complete XML specification with explicit layout hierarchies, resource bindings, and constraints. Migration therefore requires faithfully reusing this semantic information and mapping it into idiomatic target constructs, rather than inferring structure from pixels. Because the inputs, available information, and evaluation criteria differ substantially, a direct head-to-head comparison in the main evaluation would be inappropriate. Instead, we provide an exploratory comparison in the Discussion 6.2 to examine their complementary strengths.

5.2 RQ1: How is the quality of the UI migrated by GUIMIGRATOR?

We evaluate migration quality using two complementary metrics: (1) *migration completeness*, i.e., the proportion of UI elements successfully transpiled; and (2) *migration correctness*, i.e., whether the generated code compiles and renders the intended layout without breaking semantics.

Migration Completeness. We define completeness as the *migration rate*, the ratio of migrated UI elements (XML files, layouts, and views) to the total number of elements in the original Android project (Table 2). As shown in Table 3, GUIMIGRATOR consistently achieves high coverage across all ten domains. For Jetpack Compose, migration rates reach 92.0%, 95.5%, and 93.5% for XML files, layouts, and views, respectively. For SwiftUI, the corresponding rates are 90.6%, 94.5%, and 90.1%. The slightly lower SwiftUI coverage reflects its greater structural mismatch with Android XML. Per-domain analysis (Figure 7) further shows that domains dominated by standard layouts (e.g., health) achieve near-complete migration, while those relying heavily on custom widgets (e.g., business) exhibit lower coverage. Manual inspection shows that these omissions fall into

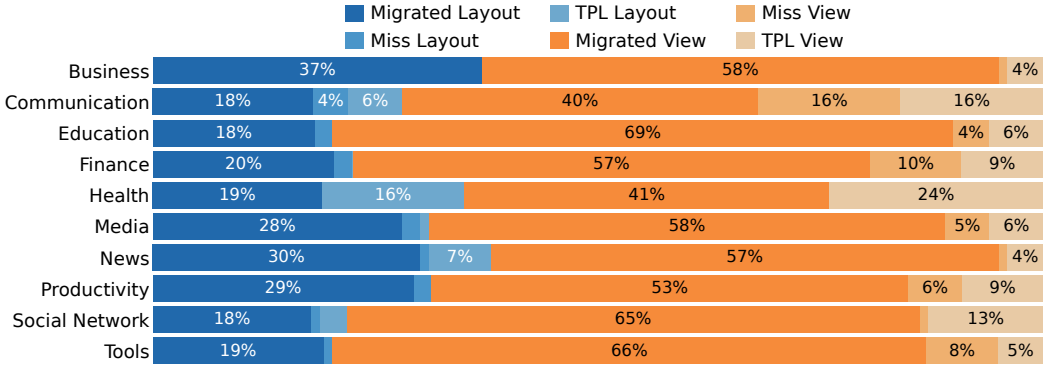


Fig. 7. Proportion of layouts and views migrated by GUIMigrator within an app, as well as the proportion of third-party library UI layouts and view types within the app.

three categories: (i) third-party or proprietary widgets without declarative counterparts, (ii) missing or dynamically generated resources, and (iii) non-standard XML constructs that violate layout specifications. These cases are explicitly recorded in the migration report and typically require only localized adjustments.

Migration Correctness. Executability Correctness is assessed by compiling the generated code and validating its rendering in native preview tools. We recorded 43 syntax errors in Compose and 61 in SwiftUI. Roughly 70% of these errors are due to unresolved external resources (e.g., missing image assets), while the remainder involve unsupported components such as `MultiAutoCompleteTextView`, which lacks a declarative counterpart. In such cases, placeholders are inserted and issues logged in a migration report to guide developer refinement.

Overall, GUIMigrator achieves migration rates above 90% on both targets, and the majority of errors are localized to unsupported or missing resources rather than systematic failures. These results indicate that GUIMigrator preserves most UI structures across diverse domains and produces compilable, visually faithful code with minimal manual intervention. This provides strong evidence that semantics-preserving transpilation is both practical and effective for cross-platform UI migration.

5.3 RQ2: How effective is GUIMigrator in preserving UI fidelity and reducing developer effort?

A central goal of migration is to preserve the visual consistency of the original UI while minimizing the amount of manual refinement required by developers. Accordingly, we evaluate effectiveness along two dimensions: (1) *visual fidelity*, measured by the Structural Similarity Index (SSIM) between screenshots of original and migrated previews. To complement this pixel-level measure, we introduce *Project-wide Semantic Coherence (PSC)*, which evaluates whether the migrated UI preserves key structural semantics beyond visual similarity. PSC compares the source and migrated interfaces on their normalized semantic graphs along three dimensions: (S) consistency of component hierarchy, (A) preservation of normalized key attributes (e.g., text, size, color, and resource bindings), and (L) retention of essential layout relations such as containment, alignment, and proportional spacing. We compute $PSC = \frac{S+A+L}{3}$; and (2) *manual adjustment effort*, quantified by Code Relative Change (CRC) and Code Token Change (CTC), which capture the extent of post-migration edits needed for compilability and visual correctness.

Table 4. Migration effort across ten domains. CRC and CTC quantify post-migration code adjustment cost; lower values are better.

Domain	Migration to Jetpack Compose						Migration to SwiftUI					
	ChatGPT-5.2		DeepSeek-V3.2		GUIMIGRATOR		ChatGPT-5.2		DeepSeek-V3.2		GUIMIGRATOR	
	CRC	CTC	CRC	CTC	CRC	CTC	CRC	CTC	CRC	CTC	CRC	CTC
Business	2.71	0.58	2.93	0.64	2.32	0.39	2.89	0.52	3.08	0.59	2.41	0.18
Comm.	2.64	0.51	2.88	0.57	2.90	0.40	1.48	0.31	1.66	0.36	4.72	0.21
Edu.	4.38	0.33	4.61	0.39	3.85	0.26	5.12	0.24	5.36	0.29	4.08	0.19
Finance	2.44	0.27	2.63	0.31	2.86	0.16	3.21	0.30	3.39	0.35	2.19	0.11
Health	2.31	0.61	2.55	0.68	1.82	0.37	1.96	0.55	2.14	0.63	1.91	0.10
Media	2.73	0.36	2.95	0.41	2.49	0.18	2.58	0.29	2.77	0.33	1.96	0.07
News	1.94	0.40	2.16	0.46	2.31	0.30	1.77	0.33	1.95	0.37	2.46	0.09
Prod.	3.62	2.61	3.81	2.74	4.08	2.54	3.55	2.68	3.72	2.82	3.97	2.49
Network	2.22	0.19	2.39	0.23	1.92	0.12	1.98	0.15	2.14	0.19	1.70	0.07
Tools	1.82	0.10	2.01	0.13	1.26	0.05	1.63	0.09	1.79	0.12	0.99	0.03
Avg	2.68	0.56	2.89	0.63	2.58	0.38	2.72	0.55	2.90	0.61	2.47	0.34

Manual Adjustment Effort. For code-level corrections, GUIMIGRATOR achieves CRC/CTC values of 2.88%/0.50% on Compose and 2.85%/0.32% on SwiftUI, meaning that over 97% of the generated code is directly usable without modification. The LLM baselines (ChatGPT-5.2 and DeepSeek-V3.2) yield numerically comparable CRC/CTC values, indicating that they can often produce syntactically plausible UI code. However, these aggregate metrics mask a key difference in error characteristics: LLM-generated outputs often require repeated localized adjustments to restore cross-file semantic consistency, which accumulates as post-migration effort. By contrast, GUIMIGRATOR integrates platform-specific constraints into transpilation, ensuring that even if placeholders are needed, the output remains compilable and extensible.

Visual Fidelity. To further understand migration fidelity, we additionally analyze both visual and semantic preservation using complementary metrics. Residual mismatches largely stem from (i) platform-specific style differences between Android and iOS, (ii) device-level variations in resolution, and (iii) unsupported third-party components replaced by placeholders. These cases are systematically logged in migration reports, enabling developers to selectively refine critical components.

Table 5 reports SSIM and PSC, which capture complementary aspects of migration fidelity. LLM-based approaches achieve SSIM scores comparable to GUIMIGRATOR (Compose: 73.1% for ChatGPT-5.2 and 72.0% for DeepSeek-V3.2 vs. 81.9% for GUIMIGRATOR; SwiftUI: 66.1% and 64.9% vs. 78.2%), indicating that modern models can generate visually plausible layouts. However, their PSC scores are consistently lower (Compose: 73.1% and 72.0% vs. 93.2% for GUIMIGRATOR) and show greater variation across domains. This difference arises because LLMs synthesize UI code in a file-local manner without enforcing a shared semantic representation, which can lead to inconsistencies in layout hierarchy, resource binding, and component relationships. In contrast, GUIMIGRATOR derives target code from an explicit semantic intermediate representation, enabling preservation of structural and cross-component semantics throughout the migration process.

Overall, GUIMIGRATOR consistently delivers higher visual fidelity and similar or lower adjustment effort. These results demonstrate that semantics-guided transpilation provides a more reliable and developer-friendly foundation for cross-platform UI migration than ad-hoc LLM prompting.

Table 5. Semantic preservation quality across ten domains. SSIM measures visual similarity, while PSC evaluates project-level structural consistency. Higher values are better.

Domain	Migration to Jetpack Compose						Migration to SwiftUI					
	ChatGPT-5.2		DeepSeek-V3.2		GUIMIGRATOR		ChatGPT-5.2		DeepSeek-V3.2		GUIMIGRATOR	
	SSIM	PSC	SSIM	PSC	SSIM	PSC	SSIM	PSC	SSIM	PSC	SSIM	PSC
Business	72.6	71.2	71.4	68.4	74.5	91.5	64.1	69.3	63.3	66.1	68.4	88.2
Comm.	72.9	70.6	72.0	67.9	73.8	89.8	70.2	68.2	69.0	65.5	72.9	86.7
Edu.	70.1	69.7	69.2	66.9	74.1	92.4	53.8	63.4	52.7	60.8	73.2	90.3
Finance	78.0	73.5	76.9	70.2	80.7	93.1	68.9	70.4	67.6	67.8	80.6	91.6
Health	67.4	75.1	66.3	72.4	78.5	95.2	64.5	72.9	63.4	70.5	77.6	93.8
Media	56.2	64.8	55.1	61.9	78.5	90.6	50.7	60.7	49.6	57.3	76.6	88.9
News	75.2	74.2	74.0	71.3	84.9	94.0	70.4	71.1	69.1	68.6	82.6	92.7
Prod.	90.6	78.5	89.1	75.9	94.2	95.8	84.7	74.2	83.2	71.6	91.3	93.4
Network	66.8	73.9	65.9	71.2	80.1	93.5	62.7	70.5	61.5	67.9	78.6	91.2
Tools	80.9	79.4	79.6	76.8	82.8	96.4	71.3	75.8	69.8	72.9	75.0	94.1
Avg	73.1	73.1	72.0	70.3	81.9	93.2	66.1	69.7	64.9	67.0	78.2	91.1

Manual check. To ensure that automated metrics reflect actual migration correctness, we manually inspected all migrated layouts produced in our evaluation. Each case was examined against three criteria derived from our semantic model: (i) preservation of view hierarchy and containment, (ii) consistency of key attributes (e.g., text, size, and bindings), and (iii) retention of essential layout relations such as alignment and weight distribution. The inspection confirmed that layouts with high PSC scores consistently satisfied these semantic checks, whereas lower-PSC cases typically involved localized degradations such as flattening nested containers or approximating weighted layouts with heuristic spacers. For example, in several generative transformations a `LinearLayout` with proportional weights (0.3/0.7) was visually reproduced but structurally rewritten as independent stacks, preserving appearance while altering layout semantics. This check supports that PSC and related metrics align with human judgment of migration correctness.

5.4 RQ3: How does GUIMIGRATOR reduce manual development effort in UI transpilation?

To evaluate the practical benefits of GUIMIGRATOR for developers, we conducted a controlled user study comparing manual SwiftUI development against GUIMIGRATOR-assisted migration. Due to the prohibitive cost of migrating all 31 applications manually, we randomly sampled 40 XML files drawn from 15 applications across eight domains, comprising 138 layouts and 368 view components (3,948 lines of UI code).

We recruited four professional mobile developers (3–5 years of experience, proficient in SwiftUI) and divided them into two groups: *Experimental Group* (2 participants), who used GUIMIGRATOR to generate SwiftUI code and only corrected migration errors; and *Control Group* (2 participants), who implemented the same UIs manually in SwiftUI from scratch. Both groups were provided with the Android source code, UI screenshots, and pre-adapted iOS resources to ensure fairness.

Table 6. Time costs and proportion of each step in UI migration by GUIMIGRATOR across dataset domains.

Group	LOC Changed	Time Cost
Control	5,294	412 min
Experimental	82	38 min

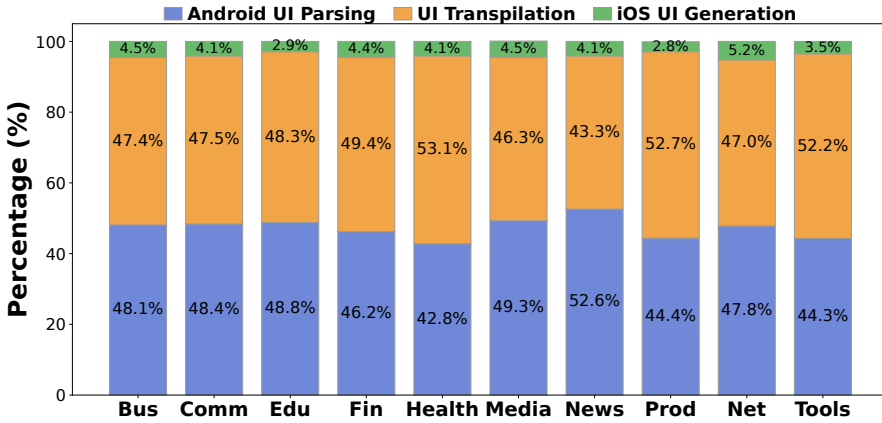


Fig. 8. Time costs and proportion of each step in UI migration by GUIMIGRATOR on each dataset domain.

Each participant was required to produce SwiftUI code rendering an iOS interface visually consistent with the original Android design. The experimental group validated and refined GUIMIGRATOR's generated code, while the control group manually wrote all layout/view code and migrated resources by hand. We verified the final code in Xcode to ensure visual fidelity.

Table 6 reports development effort in terms of *time cost* and *lines of code (LOC) changed*, two widely used proxies for developer effort. On average, the experimental group completed each interface with only 82 LOC changes and 38 minutes of work, mainly involving localized refinements such as reconnecting project-specific resources, replacing unsupported widgets with native equivalents, and minor style adjustments rather than rewriting layout structures. In contrast, the control group required 5,294 new lines of SwiftUI code and 412 minutes in total. This corresponds to over **90% reduction in both code churn and development time** when using GUIMIGRATOR. The dramatic efficiency gains stem from three factors: (i) GUIMIGRATOR automates resource parsing and adaptation, removing the need for manual asset migration; (ii) semantic transpilation generates structurally faithful SwiftUI code, minimizing manual layout reconstruction; and (iii) error handling localizes required fixes to a small set of unsupported components.

Although minor manual adjustments remain necessary, they are significantly less costly than reimplementing entire UIs. GUIMIGRATOR reduces development time and effort by an order of magnitude compared with manual development, demonstrating its strong practicality for cross-platform UI migration in real-world scenarios.

5.5 RQ4: What is the runtime efficiency of GUIMIGRATOR in UI migration?

To evaluate performance, we measure the end-to-end migration time of GUIMIGRATOR across all 31 applications, and break down the cost into three stages: (i) UI parsing and resource adaptation, (ii) semantic transpilation, and (iii) target code generation. For comparison, we also record the time required by a GPT-5-based LLM workflow, including both API invocation and inference latency.

Figure 8 reports the distribution of runtime costs across domains. On average, GUIMIGRATOR migrates one complete application in **8.1 seconds**, showing that the pipeline is lightweight and efficient. Among the three stages, semantic transpilation accounts for the largest portion (~49%), followed by parsing and resource adaptation (~47%), while code generation contributes less than 4%. The overall runtime scales linearly with the number of XML files and views, confirming the predictable behavior of the rule-based transformation process.

In contrast, the GPT-5-based workflow requires substantially more time per application, mainly due to inference latency and repeated API interactions. This highlights the efficiency advantage of a compiled transpilation pipeline for application-scale migration. Overall, GUIMIGRATOR achieves efficient migration at the application level, with low runtime overhead and predictable scaling, making it suitable for integration into real-world development workflows.

6 Discussion

6.1 LLMs for GUI Migration

Our experiments with recent LLMs (ChatGPT-5.2 and DeepSeek-V3.2) show both the potential and limitations of generative approaches for cross-platform UI migration. On the positive side, LLMs achieve moderate visual similarity, confirming their ability to generate plausible UI code given natural language prompts and structured inputs. However, we also observe several characteristic failure modes: (i) Structural drift. LLMs often restructure layouts in unintended ways, such as splitting elements into submodules or introducing undeclared variables, which reduces fidelity and can trigger compilation errors. (ii) Stylistic hallucination. LLMs may autonomously add modifiers (e.g., rounded corners, shadows) not present in the source, thereby altering the intended appearance. (iii) Syntax instability. Generated code may reference unsupported properties or undefined components, necessitating manual correction.

In contrast, GUIMIGRATOR takes a deterministic, semantics-preserving approach. By explicitly modeling layout semantics and applying platform-specific transformations, it avoids the probabilistic drift of LLMs, yielding higher structural fidelity and more stable code generation.

6.2 Exploratory Comparison with Generative Migration Approaches

To understand the applicability of LLM-based agents and multimodal screenshot-to-code models to UI migration, we conducted a small-scale exploratory study to evaluate whether these generative approaches preserve the structural semantics required for migration. We compared GUIMIGRATOR with two representative paradigms: (1) an agent-based workflow using Codex-5.2 [3], where XML layouts and referenced resources were provided to generate Compose and SwiftUI code; and (2) a vision-driven workflow using a multimodal ChatGPT-5.2 [2], where original Android UI screenshots were given as input to generate corresponding implementations in the target declarative framework. We sampled five projects and ten representative layouts per project (100 layouts total) and evaluated outputs using the same metrics as RQ2 (SSIM, PSC, and compilation validation).

From Table 7, we can see that both generative approaches produce visually plausible interfaces, achieving SSIM comparable to GUIMIGRATOR. However, their PSC scores are lower, indicating weaker preservation of component structure, attribute bindings, and layout relationships. This behavior likely stems from the fact that generative models synthesize UI code without enforcing a shared cross-file semantic representation, whereas GUIMIGRATOR performs semantic transpilation driven by an explicit intermediate representation. These observations indicate that generative synthesis and semantic transpilation serve complementary roles: LLM-based approaches enable rapid UI reconstruction, while transpilation provides stronger guarantees of structural consistency required for reliable large-scale migration.

Table 7. Exploratory comparison with generative approaches on a sampled subset (100 layouts).

Approach	SSIM	PSC	Err.
Codex-5.2	0.81	0.72	3
ChatGPT-5.2 (vision)	0.84	0.65	6
GUIMIGRATOR	0.83	0.89	5

6.3 Extensibility and Maintenance Cost

A common concern for migration approaches is whether supporting new UI constructs incurs substantial manual effort as frameworks evolve. To assess this in practice, we implemented support for additional UI features beyond the original prototype, including modern layout patterns, interaction widgets, and less common attribute semantics. As summarized in Table 8, these extensions required only localized changes at the adapter and rendering layers, typically on the order of tens of lines of code per category. No modification to the intermediate representation, resource normalization, or core transpilation pipeline was required.

These observations indicate that extending GUIMIGRATOR primarily involves refining how existing semantic roles are instantiated for new components, rather than introducing new translation mechanisms. When unsupported widgets are encountered, GUIMIGRATOR pre-

Table 8. Effort for extending GUIMIGRATOR to additional UI constructs.

Category	Added LOC	Representative Extensions
Layout	34 LOC	MotionLayout, CollapsingToolbarLayout
View	41 LOC	RecyclerView, NavigationRailView, SearchView
Attribute	27 LOC	foregroundGravity, measureWithLargestChild

serves structural layout information through a fallback representation, allowing developers to perform lightweight post-adjustments without disrupting the migrated hierarchy.

6.4 Threats to Validity

Internal validity. Our evaluation focuses primarily on UI components provided by the official Android framework. For uncommon or advanced components (e.g., TextureView for GPU-intensive apps), GUIMIGRATOR currently substitutes placeholders rather than providing exact transpilation. While this may slightly underestimate migration fidelity, the underlying transpilation architecture is modular and allows developers to extend coverage by implementing custom rules for such components. **External validity.** The generalizability of our results may be affected by applications that rely heavily on third-party or customized UI components. When no equivalent construct exists in the target platform (e.g., `com.kyleduo.switchbutton.SwitchButton`, which supports multi-state animations not directly available in SwiftUI), GUIMIGRATOR simplifies the component to the closest standard alternative and logs the case in a migration report. This ensures functional completeness but may reduce visual fidelity. Although this limitation is inherent to cross-platform UI differences, our reporting mechanism enables developers to identify and refine such cases manually. We mitigate these threats by (i) selecting a diverse dataset covering multiple domains, (ii) explicitly reporting unsupported components and their locations, and (iii) designing the transpiler to be easily extensible to new UI elements. Together, these measures reduce the risk of overstating the effectiveness of GUIMIGRATOR.

7 Related Work

Cross-platform Mobile Development Frameworks. Cross-platform development frameworks have become increasingly popular, facilitating the creation of mobile applications that can run on multiple platforms, e.g., Android and iOS. [19–22, 27–29, 31, 34, 35, 44, 47]. Wafaa S. et al. [27] developed a novel code transformation technique leveraging XSLT and regular expressions. This approach aimed to streamline the cross-platform mobile development process, facilitating the conversion of applications from Windows Phone 8 to Android. Henning et al. [31] introduced MD2, a model-driven approach for cross-platform application development. This approach allowed developers to define applications using a concise domain-specific language (DSL). From the abstract

model, the system could automatically generate applications for both Android and iOS. However, while cross-platform technologies offer certain advantages, they still fall short compared to native development, particularly in terms of performance, support for platform-specific features, and application size.

Screenshot-Based UI Generation. Generating UI code from screenshots or UI design diagrams is a popular research topic [18, 23–25, 38, 42, 43, 49, 52]. Typically, these approaches use computer vision to identify the UI structure within images and leverage deep learning techniques to classify UI elements, thereby constructing the corresponding UI code. Beltramelli et al. [18] introduced *pix2code*, an approach based on convolutional and recurrent neural networks. Their approach took a single screenshot as input and generated the GUI using a domain-specific language (DSL) to describe UI elements and reduce the search space. While *pix2code* generated iOS UIs based on Storyboard from screenshots, it could not produce declarative code. Natarajan et al. [42] presented the P2A tool, which combined optical character recognition (OCR) technology and heuristic rules to identify UI elements in Android screenshots. Screenshot-based UI generation technologies were constrained by the accuracy of computer vision in recognizing images, making it challenging to handle complex graphical structures. Additionally, deep learning-based approaches require substantial training data, which is difficult to obtain for the iOS platform due to the lack of open-source data. Furthermore, no current technology can generate declarative UI structures. GUIMIGRATOR addresses this gap by using migration techniques to facilitate cross-platform reuse of UI.

UI Testing Migration. There is extensive research on the migration of mobile testing cases [15–17, 32, 36, 37, 46, 50]. They aim to enable the reuse of test cases across applications with the same or similar functionalities, which can include cross-platform scenarios. By leveraging existing test cases for new or related applications, these approaches seek to optimize testing processes and improve efficiency, as well as reduce redundancy and save efforts. Mariani et al. [41] framed the problem of test reuse as a search challenge, employing evolutionary testing to facilitate test migration between different Android applications. TestMig [46] is a migration tool designed to transfer tests from iOS to Android by leveraging UI events. This tool requires access to the source code of both the source and target applications. Compared to the migration of UI test cases, our GUIMIGRATOR focuses on the migration of the UI code itself, enabling cross-platform reuse of UI components.

8 Conclusion and Future Work

We presented GUIMIGRATOR, a semantics-preserving approach for mobile UI migration that enables the reuse of Android XML-based interfaces on declarative frameworks such as SwiftUI and Jetpack Compose. Our evaluation across 31 applications from ten domains shows that GUIMIGRATOR achieves high migration completeness and visual fidelity, while reducing manual development effort by more than an order of magnitude compared with manual coding, and outperforming LLM-based baselines. In future work, we plan to broaden the scope of migration by supporting bidirectional transformations and additional platforms (e.g., Flutter, React Native). We further plan to extend GUIMIGRATOR to support richer UI semantics (e.g., animations, accessibility tags, interaction logic, and dynamically constructed UI elements in Java/Kotlin through snapshot or runtime analysis).

9 Data Availability

Our tool is available at [6].

Acknowledgments

This work was supported by the National Key R&D Program of China (No. 2024YFB4506400).

References

- [1] 2026. Android. <https://www.android.com/>.
- [2] 2026. ChatGPT. <https://chatgpt.com/>.
- [3] 2026. Codex. <https://openai.com/codex/>.
- [4] 2026. Compose. <https://developer.android.com/compose/>.
- [5] 2026. Flutter. <https://flutter.dev/>.
- [6] 2026. GUIMigrator. <https://github.com/testmigrator/guimigrator>.
- [7] 2026. GUIMigrator Datasets. <https://github.com/testmigrator/guimigrator/tree/master/experiments>.
- [8] 2026. iOS. <https://developer.apple.com/ios/>.
- [9] 2026. open-source-android-apps. <https://github.com/pcqpcq/open-source-android-apps>.
- [10] 2026. OpenShop. <https://github.com/openshopio/openshop.io-android/>.
- [11] 2026. ReactNative. <https://reactnative.dev/>.
- [12] 2026. SwiftUI. <https://developer.apple.com/cn/xcode/swiftui/>.
- [13] Hamza Abbas, Sehrish Munir, Muhammad Tahir, Misbah Noor, Qaisra Honey, M Ameer Hamza, and Muhammad Waseem Iqbal. 2024. Enhancing User Experience: Cross-Platform Usability in Digital Banking Apps. *Journal of Computing & Biomedical Informatics* 7, 02 (2024).
- [14] Faiz Ahmed, Xuchen Tan, Folajinmi Adewole, Suprakash Datta, and Maleknaz Nayebi. 2025. Inferring questions from programming screenshots. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 750–755. doi:10.1109/MSR66628.2025.00111
- [15] Farnaz Behrang and Alessandro Orso. 2018. Automated test migration for mobile apps. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*. 384–385. doi:10.1145/3183440.3195019
- [16] Farnaz Behrang and Alessandro Orso. 2019. Test migration between mobile apps with similar functionality. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 54–65. doi:10.1109/ASE.2019.00016
- [17] Farnaz Behrang and Alessandro Orso. 2020. AppTestMigrator: a tool for automated test migration for Android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 17–20. doi:10.1145/3377812.3382149
- [18] Tony Beltramelli. 2018. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems*. 1–6. doi:10.1145/3220134.3220135
- [19] Andreas Björn-Hansen and Gheorghita Ghinea. 2018. Bridging the gap: Investigating device-feature exposure in cross-platform development. (2018). <https://hdl.handle.net/10125/50605>
- [20] Andreas Björn-Hansen, Tor-Morten Grønli, and Gheorghita Ghinea. 2018. A Survey and Taxonomy of Core Concepts and Research Challenges in Cross-Platform Mobile Development. *ACM Computing Surveys (CSUR)* 51 (2018), 1 – 34. <https://api.semanticscholar.org/CorpusID:59337596>
- [21] Andreas Björn-Hansen, Christoph Rieger, Tor-Morten Grønli, Tim A Majchrzak, and Gheorghita Ghinea. 2020. An empirical investigation of performance overhead in cross-platform mobile development frameworks. *Empirical Software Engineering* 25 (2020), 2997–3040. doi:10.1007/S10664-020-09827-6
- [22] Sanchit Chadha, Antuan Byalik, Eli Tilevich, and Alla Rozovskaya. 2017. Facilitating the development of cross-platform software via automated code synthesis from web-based programming resources. *Computer Languages, Systems & Structures* 48 (2017), 3–19. doi:10.1016/J.CL.2016.08.005
- [23] Chunyang Chen, Ting Su, Guozhu Meng, Zhenchang Xing, and Yang Liu. 2018. From ui design image to gui skeleton: a neural machine translator to bootstrap mobile gui implementation. In *Proceedings of the 40th International Conference on Software Engineering*. 665–676. doi:10.1145/3180155.3180240
- [24] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. 2019. Storydroid: Automated generation of storyboard for Android apps. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 596–607. doi:10.1109/ICSE.2019.00070
- [25] Sen Chen, Lingling Fan, Ting Su, Lei Ma, Yang Liu, and Lihua Xu. 2019. Automated cross-platform GUI code generation for mobile apps. In *2019 IEEE 1st International Workshop on Artificial Intelligence for Mobile (AI4Mobile)*. IEEE, 13–16.
- [26] Zirui Chen, Xing Hu, Xin Xia, and Xiaohu Yang. 2026. Every Maintenance Has Its Exemplar: The Future of Software Maintenance through Migration. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.48550/ARXIV.2602.14046
- [27] Wafaa S El-Kassas, Bassem A Abdullah, Ahmed H Yousef, and Ayman M Wahba. 2016. Enhanced code conversion approach for the integrated cross-platform mobile development (icpmd). *IEEE Transactions on Software Engineering* 42, 11 (2016), 1036–1053. doi:10.1109/TSE.2016.2543223
- [28] Wafaa S El-Kassas, Bassem A Abdullah, Ahmed H Yousef, and Ayman M Wahba. 2017. Taxonomy of cross-platform mobile applications development approaches. *Ain Shams Engineering Journal* 8, 2 (2017), 163–190.
- [29] El Hassane Ettifouri, Abdelkader Rhouati, Jamal Berrich, and Toumi Bouchentouf. 2017. Toward a merged approach for cross-platform applications (web, mobile and desktop). In *Proceedings of the 2017 International Conference on Smart*

- Digital Environment*. 207–213. doi:10.1145/3128128.3128160
- [30] Sidong Feng, Minmin Jiang, Tingting Zhou, Yankun Zhen, and Chunyang Chen. 2022. Auto-icon+: An automated end-to-end code generation tool for icon designs in ui development. *ACM Transactions on Interactive Intelligent Systems* 12, 4 (2022), 1–26. doi:10.1145/3531065
 - [31] Henning Heitkötter, Tim A Majchrzak, and Herbert Kuchen. 2013. Cross-platform model-driven development of mobile applications with md2. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing*. 526–533.
 - [32] Ruihua Ji, Tingwei Zhu, Xiaoqing Zhu, Chunyang Chen, Minxue Pan, and Tian Zhang. 2023. Vision-Based Widget Mapping for Test Migration Across Mobile Platforms: Are We There Yet?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1416–1428. doi:10.1109/ASE56229.2023.00068
 - [33] Yilei Jiang, Yaozhi Zheng, Yuxuan Wan, Jiaming Han, Qunzhong Wang, Michael R Lyu, and Xiangyu Yue. 2025. ScreenCoder: Advancing Visual-to-Code Generation for Front-End Automation via Modular Multimodal Agents. *arXiv preprint arXiv:2507.22827* (2025). doi:10.48550/ARXIV.2507.22827
 - [34] Parsa Karami, Ikram Darif, Cristiano Politowski, Ghizlane El-Boussaidi, Sègla Kpodjedo, and Imen Benzarti. 2023. On the Impact of Development Frameworks on Mobile Apps. *2023 30th Asia-Pacific Software Engineering Conference (APSEC)* (2023), 131–140. <https://api.semanticscholar.org/CorpusID:268878306>
 - [35] Mohamed Lachgar, Mohamed Hanine, Hanane Benouda, and Younes Ommame. 2022. Decision Framework for Cross-Platform Mobile Development Frameworks Using an Integrated Multi-Criteria Decision-Making Methodology. *Int. J. Mob. Comput. Multim. Commun.* 13 (2022), 1–22. <https://api.semanticscholar.org/CorpusID:247118680>
 - [36] Jun-Wei Lin, Reyhaneh Jabbarvand, and Sam Malek. 2019. Test transfer across mobile apps through semantic mapping. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 42–53. doi:10.1109/ASE.2019.00015
 - [37] Jun-Wei Lin and Sam Malek. 2022. Gui test transfer from web to android. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 1–11. doi:10.1109/ICST53961.2022.00011
 - [38] Yanbin Liu, Qidi Hu, and Kunxian Shu. 2018. Improving pix2code based Bi-directional LSTM. In *2018 IEEE International Conference on Automation, Electronics and Electrical Engineering (AUTEEL)*. IEEE, 220–223.
 - [39] Yuwen Lu, Alan Leung, Amanda Swearngin, Jeffrey Nichols, and Titus Barik. 2025. Misty: Ui prototyping through interactive conceptual blending. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–17. doi:10.1145/3706598.3713924
 - [40] Amira T Mahmoud, M Radwan, Abdelrahman Mohamed Soliman, AH Youssef, Hala H Zayed, and Walaa Medhat. 2024. Trans-Compiler-Based Conversion from Cross-Platform Applications to Native Applications. *Ann. Emerg. Technol. Comput.(AETiC)* 8 (2024).
 - [41] Leonardo Mariani, Mauro Pezzè, and Daniele Zuddas. 2018. Augusto: Exploiting popular functionalities for the generation of semantic gui tests with oracles. In *Proceedings of the 40th International Conference on Software Engineering*. 280–290. doi:10.1145/3180155.3180162
 - [42] Siva Natarajan and Christoph Csallner. 2018. P2A: A tool for converting pixels to animated mobile application user interfaces. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*. 224–235. doi:10.1145/3197231.3197249
 - [43] Tuan Anh Nguyen and Christoph Csallner. 2015. Reverse engineering mobile application user interfaces with remaui (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 248–259. doi:10.1109/ASE.2015.32
 - [44] Robin Nunkesser. 2018. Beyond web/native/hybrid: A new taxonomy for mobile app development. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*. 214–218. doi:10.1145/3197231.3197260
 - [45] Diego Geovanny Falconi Punguil, David Mauricio Cordonez Arias, Edwin Fabricio Damacela Calero, and Edison Isaías Pallo Cuchiparte. 2025. Análisis comparativo de la productividad en el desarrollo móvil: Flutter vs Kotlin vs Swift. In *ASCE*, Vol. 4. 1301–1316.
 - [46] Xue Qin, Hao Zhong, and Xiaoyin Wang. 2019. Testmig: Migrating gui test cases from ios to android. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 284–295. doi:10.1145/3293882.3330575
 - [47] Christoph Rieger and Tim A. Majchrzak. 2019. Towards the definitive evaluation framework for cross-platform app development approaches. *J. Syst. Softw.* 153 (2019), 175–199. <https://api.semanticscholar.org/CorpusID:132090861>
 - [48] Juan A Rodriguez, Abhay Puri, Shubham Agarwal, Issam H Laradji, Pau Rodriguez, Sai Rajeswar, David Vazquez, Christopher Pal, and Marco Pedersoli. 2025. Starvector: Generating scalable vector graphics code from images and text. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 16175–16186. doi:10.1109/CVPR52734.2025.01508
 - [49] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *arXiv preprint arXiv:2403.03163* (2024). doi:10.48550/ARXIV.2403.03163
 - [50] Saghar Talebipour, Yixue Zhao, Luka Dojčilović, Chenggang Li, and Nenad Medvidović. 2021. Ui test migration across mobile platforms. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE,

756–767. doi:10.1109/ASE51524.2021.9678643

- [51] Vanja Vidmark. 2025. Performance Evaluation of Kotlin Multiplatform on iOS: Quantitative comparison between Kotlin and Compose Multiplatform against Native Swift for UI Rendering Hardware Interactions.
- [52] Shuhong Xiao, Yunnong Chen, Jiazhi Li, Liuqing Chen, Lingyun Sun, and Tingting Zhou. 2024. Prototype2Code: End-to-end Front-end Code Generation from UI Design Prototypes. *arXiv preprint arXiv:2405.04975* (2024). doi:10.48550/ARXIV.2405.04975
- [53] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. 2024. LlamaTouch: A Faithful and Scalable Testbed for Mobile UI Automation Task Evaluation. *arXiv preprint arXiv:2404.16054* (2024). doi:10.48550/ARXIV.2404.16054

Received 2026-02-23; accepted 2026-03-24